UNIVERSIDAD NACIONAL DEL ALTIPLANO PUNO

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

SEMESTRE VIII - GRUPO A - 2026 - II - UNIDAD 1

"CLASIFICACIÓN DE CULTIVARES DE VINO MEDIANTE UNA RED NEURONAL MLP"

CURSO:

REDES NEURONALES

ALUMNOS:

APAZA MEDINA NILBER YOSEL

DOCENTE:

Dr. ROMERO FLORES ROBERT ANTONIO

FECHA DE ENTREGA:

20 DE OCTUBRE DE 2026

Resumen

Este trabajo desarrolla un clasificador supervisado para identificar el cultivar de una muestra de vino a partir de sus propiedades químicas. Se utiliza el conjunto Wine Recognition de UCI, con 178 observaciones, 13 variables numéricas y tres clases. La solución implementa una red neuronal multicapa (MLP) mediante TensorFlow/Keras. El flujo incluye partición estratificada, estandarización, entrenamiento con Adam y detención temprana y evaluación mediante exactitud, precisión, recall, F1 y matriz de confusión. En la partición de prueba de 36 muestras, el modelo clasificó correctamente 35 (exactitud 97.22 %). Este resultado es alentador, aunque debe interpretarse con cautela por el tamaño reducido de la prueba y porque representa una sola partición.

Palabras clave: clasificación supervisada, MLP, TensorFlow, Keras, vino.

Índice general

1. Introducción **1**

1.1. Objetivos 1

2. Dataset **2**

3. Fundamentos y arquitectura **4**

4. Procedimiento **5**

5. Resultados **6**

6. Conclusiones **8**

A. Explicación básica del código **10**

A.1. Código fuente en GitHub: https://github.com/NilberMedina/CLASIFICACI-N-DE-CULTIVAR

A.2. Carga y preparación 10

A.3. Separación y escalado 10

A.4. Definición y entrenamiento de la MLP 11

A.5. Evaluación 11

B. Ejecución **12**

Índice de figuras

2.1. Cantidad de observaciones por clase en el conjunto completo. 2

3.1. Arquitectura de la red MLP en TensorFlow/Keras. 4

5.1. Matriz de confusión. Filas: clase real; columnas: clase predicha. 6

5.2. Pérdida y exactitud en entrenamiento y validación. 7

Índice de tablas

2.1. Resumen del conjunto de datos 2

3.1. Hiperparámetros 4

5.1. Métricas por clase en prueba 6

Capítulo 1

Introducción

La clasificación supervisada aprende a asignar etiquetas conocidas a observaciones nuevas. En este proyecto, cada observación representa una muestra de vino descrita por mediciones químicas; la etiqueta identifica uno de tres cultivares. Es un problema multiclase y tabular que se aborda con una red neuronal densa de tipo MLP.

El objetivo es identificar cultivar/origen, no predecir la calidad sensorial del vino. El dataset Wine Recognition procede del repositorio UCI y también se distribuye en scikit-learn [1, 2]. Se emplea TensorFlow/Keras, framework presentado en el material del curso.

1.1. Objetivos

General: implementar y evaluar una MLP en TensorFlow/Keras para clasificar muestras en tres cultivares. **Específicos:** describir el dataset; preparar los datos sin fuga de información; construir y entrenar la red; analizar métricas y limitaciones.

Capítulo 2

Dataset

El conjunto contiene análisis químicos de vinos de tres cultivares de una región italiana. Tiene 178 filas completas y 13 características continuas. La etiqueta original se codifica como 0, 1 y 2, con nombres descriptivos `cultivar_0`, `cultivar_1` y `cultivar_2`. La distribución es 59, 71 y 48 muestras.

Tabla 2.1: Resumen del conjunto de datos

Propiedad	Valor
Observaciones	178
Características	13
Clases	3
Problema	Clasificación multiclase
Archivo	`wine.csv`

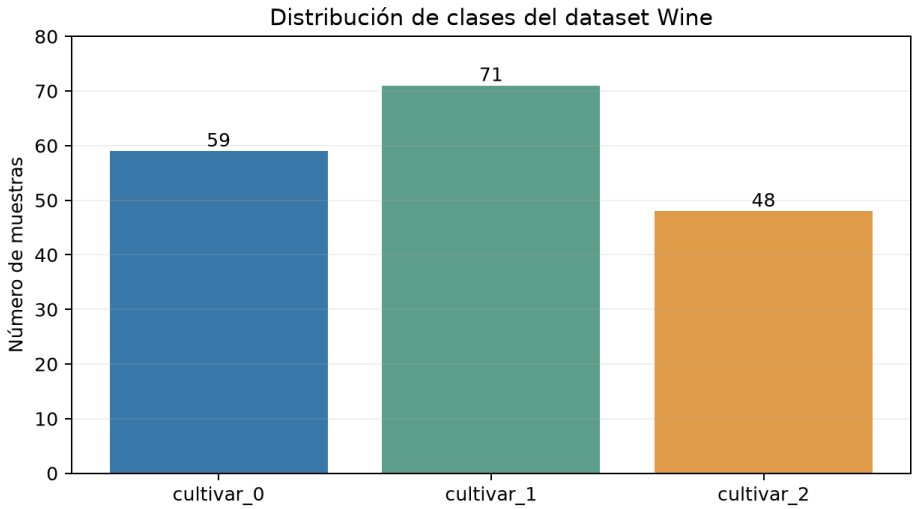


Figura 2.1: Cantidad de observaciones por clase en el conjunto completo.

Las entradas son: alcohol, ácido málico, cenizas, alcalinidad de cenizas, magnesio, fenoles totales, flavonoides, fenoles no flavonoides, proantocianinas, intensidad del color, tono,

relación OD280/OD315 de vinos diluidos y prolina. Las columnas `target` y `class_name` son etiquetas y no se usan como predictores.

Capítulo 3

Fundamentos y arquitectura

Una MLP contiene una capa de entrada, capas densas ocultas y una capa de salida. Cada neurona combina sus entradas mediante pesos y sesgo y aplica una activación. La red implementada tiene arquitectura:

$$13 \longrightarrow 32 \longrightarrow 16 \longrightarrow 3.$$

Las capas ocultas utilizan ReLU, $f(x) = \max(0, x)$. La salida usa Softmax para producir probabilidades que suman uno entre las tres clases.

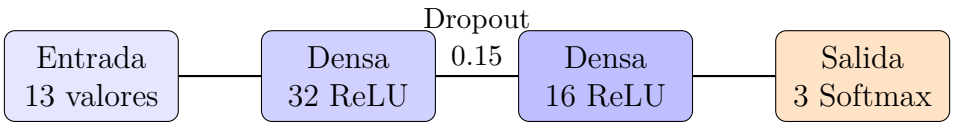


Figura 3.1: Arquitectura de la red MLP en TensorFlow/Keras.

Tabla 3.1: Hiperparámetros

Framework	TensorFlow/Keras
Capas ocultas	32 y 16 neuronas
Activaciones	ReLU; salida Softmax
Optimizador	Adam, tasa 0.001
Pérdida	`sparse_categorical_crossentropy`
Regularización	Dropout 0.15
Máximo de épocas / lote	150 / 16
Detención temprana	paciencia 12, monitorea `val_loss`
Semilla	42

Se usan etiquetas enteras con `sparse_categorical_crossentropy`. Adam actualiza los pesos a partir de gradientes por lotes. `EarlyStopping` interrumpe el ajuste si la pérdida de validación deja de mejorar y restaura los mejores pesos.

Capítulo 4

Procedimiento

El procedimiento, implementado en `main.py`, consta de:

1. Leer `wine.csv` con pandas.
2. Separar 13 entradas X de la etiqueta y.
3. Dividir 80 % entrenamiento y 20 % prueba, con estratificación y semilla fija: 142 y 36 observaciones.
4. Ajustar `StandardScaler` solo en entrenamiento y transformar ambos conjuntos.
5. Entrenar la MLP; reservar 20 % del entrenamiento para validación.
6. Evaluar sobre la prueba reservada con métricas por clase y matriz de confusión.
7. Guardar modelo, escalador, reporte y curvas de aprendizaje.

La estandarización se aplica porque las variables están en escalas distintas. Ajustar el escalador solo con entrenamiento evita que estadísticas del conjunto de prueba influyan en el aprendizaje.

La exactitud es la proporción de predicciones correctas. Precisión mide cuántas predicciones asignadas a una clase son correctas; recall mide cuántas muestras reales de esa clase fueron encontradas. F1 es su media armónica:

$$\text{Exactitud} = \frac{\text{aciertos}}{\text{total}}, \qquad F1 = 2\frac{\text{precisión}\,\text{recall}}{\text{precisión} + \text{recall}}.$$

Capítulo 5

Resultados

La ejecución con semilla 42 usó 142 observaciones para entrenamiento y 36 para prueba. Obtuvo pérdida de prueba 0.0295 y exactitud 0.9722. La detención temprana terminó tras 134 épocas.

Tabla 5.1: Métricas por clase en prueba

Clase	Precisión	Recall	F1	Soporte
cultivar_0	1.00	1.00	1.00	12
cultivar_1	0.93	1.00	0.97	14
cultivar_2	1.00	0.90	0.95	10
Exactitud global		0.9722		36

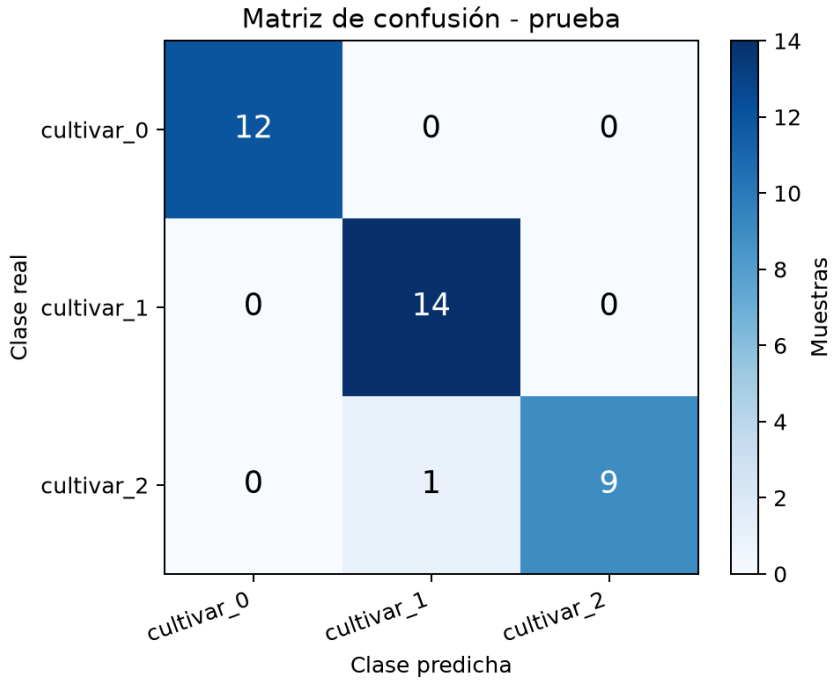


Figura 5.1: Matriz de confusión. Filas: clase real; columnas: clase predicha.

La única equivocación fue una muestra real de `cultivar_2`, predicha como `cultivar_1`. Se clasificaron correctamente las 12 muestras de cultivar 0 y las 14 de cultivar 1.

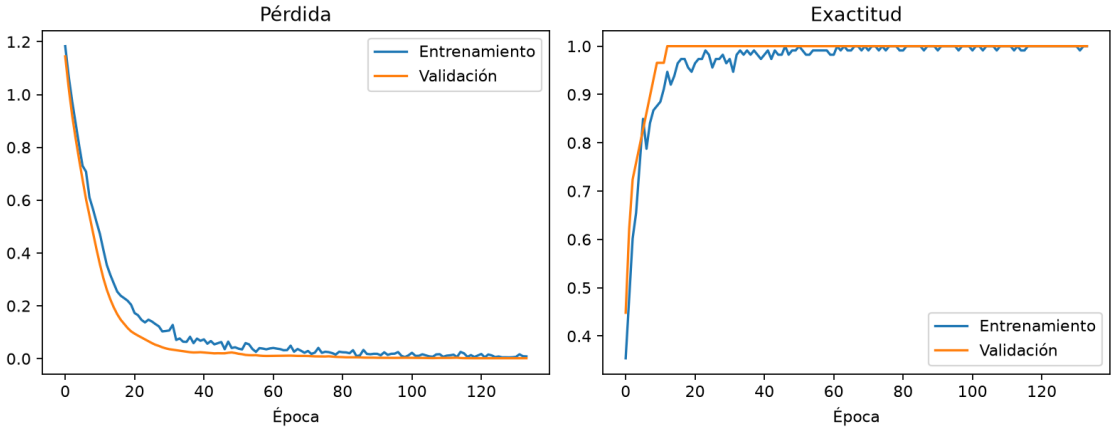


Figura 5.2: Pérdida y exactitud en entrenamiento y validación.

El resultado es alto en esta partición, pero la prueba contiene solo 36 muestras. Una predicción adicional incorrecta cambiaría la exactitud alrededor de 2.78 puntos porcentuales. Por tanto, 97.22 % no garantiza igual desempeño en muestras futuras. Se recomienda validación cruzada estratificada y reportar promedio y variabilidad.

Capítulo 6

Conclusiones

1. Se implementó una MLP multiclase con TensorFlow/Keras sobre las 178 muestras del Wine Recognition Dataset.
2. La partición estratificada y el escalado ajustado solo con entrenamiento reducen el riesgo de fuga de información.
3. En prueba se obtuvieron 35 aciertos de 36 (97.22 %); el error ocurrió entre cultivares 2 y 1.
4. El dataset y la prueba son pequeños, de modo que el resultado es una evaluación inicial y puede cambiar en otra partición.

Como trabajo futuro se recomienda usar validación cruzada estratificada, comparar configuraciones con un protocolo definido previamente y evaluar el modelo en datos externos. Las conclusiones se limitan a las tres clases de este dataset.

Bibliografía

[1] Dua, D. y Graff, C. (2019). *Wine [Dataset]*. UCI Machine Learning Repository. https://doi.org/10.24432/C5PC7J

[2] Scikit-learn developers. *load_wine: Wine recognition dataset*. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_wine.html

[3] TensorFlow. *Keras: The Python deep learning API*. https://www.tensorflow.org/guide/keras

Apéndice A

Explicación básica del código

El código realiza cuatro tareas esenciales: prepara las entradas y etiquetas, separa los datos, define la red neuronal y evalúa sus predicciones.

A.1. Código fuente en GitHub: https://github.com/NilberMedina/CLASIFICACI-N-DE-CULTIVARES-DE-VINO

A.2. Carga y preparación

Se leen las filas del CSV. Las 13 columnas químicas se guardan en X, mientras que `target` contiene la clase que el modelo debe aprender.

```
df = pd.read_csv("wine.csv")
X = df.drop(columns=["target", "class_name"]).to_numpy()
y = df["target"].to_numpy()
```

A.3. Separación y escalado

La división estratificada reserva el 20 % para la prueba y mantiene la proporción de clases. El escalador aprende la media y desviación solo del conjunto de entrenamiento; después transforma ambos conjuntos.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

A.4. Definición y entrenamiento de la MLP

La red tiene dos capas ocultas con ReLU y una salida Softmax de tres neuronas. Adam ajusta los pesos y la función de pérdida compara las predicciones con las clases correctas.

```
model = keras.Sequential([
    keras.Input(shape=(13,)),
    layers.Dense(32, activation="relu"),
    layers.Dropout(0.15),
    layers.Dense(16, activation="relu"),
    layers.Dense(3, activation="softmax"),
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(X_train, y_train, validation_split=0.20,
          epochs=150, batch_size=16,
          callbacks=[early_stop])
```

A.5. Evaluación

Finalmente, el modelo predice las clases del conjunto de prueba y se calcula la exactitud como la proporción de aciertos.

```
probabilities = model.predict(X_test)
y_pred = probabilities.argmax(axis=1)
accuracy = accuracy_score(y_test, y_pred)
```

Apéndice B

Ejecución

Desde una terminal abierta en `recursos/`, ejecutar:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
python main.py
```

El proyecto distribuido incluye dataset, código y dependencias. La ejecución crea `resultados/` con el modelo, el escalador, las métricas y las curvas.